

Le Monde des Blocs

Rapport de Projet - Intelligence Artificielle

L3 Informatique

DOSSEH Georges
KODJO Afiwa Aimée

19 novembre 2024

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs	2
1.3	Structure du code	3
1.3.1	Organisation des packages	3
1.4	Description des Classes du Package Blocksworld	3
1.4.1	Classes Principales	3
1.4.2	Classes de Démo	4
2	Modélisation	5
2.1	Représentation de l'État du Monde	5
2.2	Implémentation des Contraintes	5
2.2.1	Contraintes de Base	5
3	Algorithmes et Implémentation	6
3.1	Planification	6
3.1.1	Actions Implémentées	6
3.1.2	Heuristiques	6
3.1.3	Algorithmes de Recherche	6
3.1.4	Comparaison des Planificateurs	7
4	Satisfaction de Contraintes	9
4.1	Solveurs	9
4.2	Extraction de Connaissances	10
5	Conclusion	11

Chapitre 1

Introduction

1.1 Contexte

Le projet "Fil Rouge" que nous présentons ici est une implémentation complète du monde des blocs (BlocksWorld), un exemple classique utilisé en Intelligence Artificielle. Réalisé en Java, ce travail englobe plusieurs aspects cruciaux de la modélisation, des configurations du monde des blocs à l'extraction de connaissances à partir de bases de données. Ce dernier est utilisé notamment pour évaluer différentes approches de planification et de résolution de contraintes.

1.2 Objectifs

Les objectifs de ce projet sont :

- Implémenter une représentation efficace du monde des blocs
- Développer et comparer différents algorithmes de planification
- Créer des solveurs de contraintes pour les configurations
- Analyser des configurations via l'extraction de connaissances

1.3 Structure du code

1.3.1 Organisation des packages

Le code est organisé en plusieurs packages :

- `blocksworld` : Package principal contenant les classes spécifiques au monde des blocs.
- `planning` : Package pour les algorithmes de planification
- `cp` : Package pour les solveurs de contraintes
- `datamining` : Package pour l'extraction de connaissances
- `modelling` : Package contenant les abstractions pour la modélisation des variables et contraintes

1.4 Description des Classes du Package Blocksworld

1.4.1 Classes Principales

- **Blocksworld**
 - Classe centrale représentant l'état du monde
 - Gère les variables `freePiles`, `onBlocks`, `fixedBlocks`
 - Fournit les méthodes pour modifier l'état : `setfixed`, `setFree`, `putOn`, `putInPile`
- **BlocksworldVariables**
 - Gère la création et l'organisation des variables du monde
 - Définit les domaines des variables
 - Fournit les accesseurs pour les variables `onB`, `fixedB`, `freeP`
- **BasicActionWorld**
 - Implémente les actions possibles dans le monde
 - Hérite de `Blocksworld`
 - Construit toutes les actions valides via `buildActions()`
- **BlocksworldConstraint**
 - Vérifie les contraintes de base du monde
 - Assure qu'on ne peut pas avoir 2 blocs sur un même bloc
 - Vérifie les implications entre variables
- **IncreaseImplication**
 - Vérifie les contraintes de configuration croissante
 - S'assure qu'un bloc ne peut être posé que sur un bloc de numéro inférieur
- **RegularImplication**
 - Vérifie les contraintes de configuration régulière
 - Assure un écart constant entre les blocs d'une même pile
- **BlocksworldHeuristic & BlocksworldHeuristic2**
 - Implémentent différentes heuristiques pour A^*
 - Première basée sur le nombre de blocs
 - Seconde basée sur le nombre de piles non libres

1.4.2 Classes de Démo

`BlocksworldPlannerDemo`, `BlocksworldConstraintSolverDemo`, `BlocksworldAssociationRulesDemo`, `BlocksworldAdvancedConstraintsDemo` : Classes exécutables pour tester différents aspects :

- `BlocksworldPlannerDemo` : Tests des algorithmes de planification avec visualisation
- `BlocksworldConstraintSolverDemo` : Tests des solveurs de contraintes
- `BlocksworldAdvancedConstraintsDemo` : Tests des contraintes
- `BlocksworldAssociationRulesDemo` : Tests de l'extraction de connaissances

Chapitre 2

Modélisation

2.1 Représentation de l'État du Monde

Notre modélisation repose sur trois types de variables :

- Variables on_b : Position des blocs
 - Domaine : $\{-p, \dots, -1, 0, \dots, n-1\} \setminus \{b\}$
 - Valeurs négatives pour les piles
 - Valeurs positives pour les blocs
- Variables $fixed_b$: État de fixation (booléen)
- Variables $free_p$: Disponibilité des piles (booléen)

2.2 Implémentation des Contraintes

2.2.1 Contraintes de Base

Trois types de contraintes fondamentales :

1. Unicité des positions : deux blocs ne peuvent pas occuper la même position
2. Contraintes de fixation : un bloc avec un autre au-dessus est fixé
3. Contraintes de pile : une pile contenant un bloc n'est pas libre

Chapitre 3

Algorithmes et Implémentation

3.1 Planification

Les démonstrations de planification montrent :

- La recherche d'un plan pour déplacer un bloc d'une configuration à une autre
- La comparaison des performances entre BFS, DFS, Dijkstra et A*
- L'impact des heuristiques sur le temps de calcul

3.1.1 Actions Implémentées

Les actions sont implémentées via la classe `BasicActionWorld`. Quatre types d'actions ont été implémentés :

1. Bloc vers Bloc
2. Bloc vers Pile
3. Pile vers Bloc
4. Pile vers Pile

3.1.2 Heuristiques

Deux heuristiques ont été implémentées dans les classes `BlocksworldHeuristic` et `BlocksworldHeuristic2` :

1. `BlocksworldHeuristic` : Basée sur le nombre de blocs
2. `BlocksworldHeuristic2` : Basée sur le nombre de piles non libres

Ces heuristiques sont admissibles car elles ne surestiment jamais le coût réel.

3.1.3 Algorithmes de Recherche

Nous avons implémenté et comparé quatre algorithmes :

- BFS (Breadth-First Search)
- DFS (Depth-First Search)
- Dijkstra
- A* avec heuristiques

Configuration initiale :

Pile 1 : [0,1,2]

Pile 2 : [3,4]

L'objectif était de placer le bloc 3 sur le bloc 2.

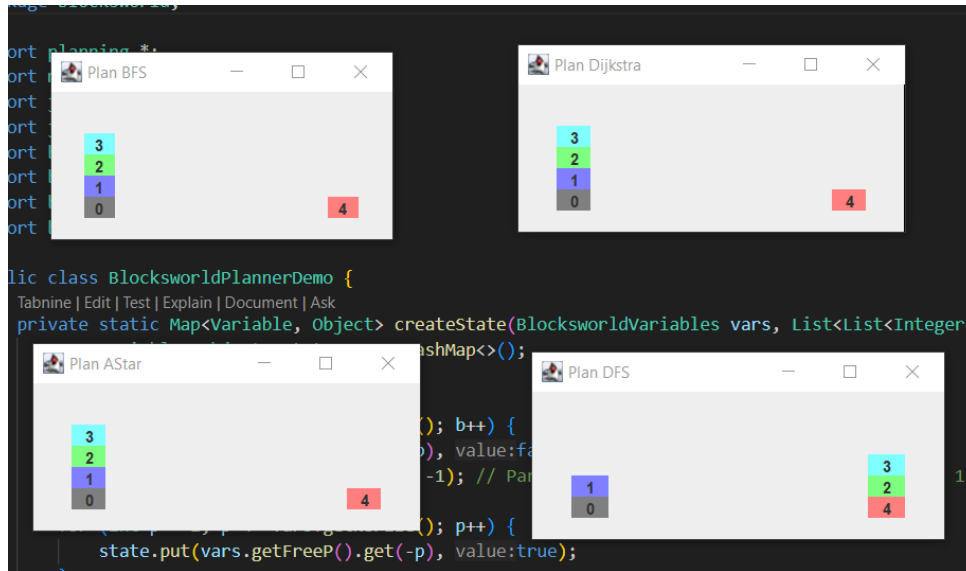


FIGURE 3.1 – Affichage des algorithmes

3.1.4 Comparaison des Planificateurs

Algorithme	Complexité	Avantages	Limitations
BFS	$O(b^d)$	Optimal	Coût mémoire élevé
DFS	$O(b^d)$	Mémoire efficace	Non optimal
A*	$O(b^d)$	Optimal et informé	Dépend de l'heuristique
Dijkstra	$O(b^d)$	Optimal en coût	Temps de calcul élevé

TABLE 3.1 – Comparaison des algorithmes de planification

```
BFS Planner:
Temps d'execution: 2 ms
Noeuds explores: 3
Nombre d'actions dans le plan: 2

Astar Planner:
Temps d'execution: 19 ms
Noeuds explores: 11
Nombre d'actions dans le plan: 2

DFS Planner:
Temps d'execution: 8 ms
Noeuds explores: 6
Nombre d'actions dans le plan: 5

Dijkstra Planner:
Temps d'execution: 336 ms
Noeuds explores: 1920
Nombre d'actions dans le plan: 2
```

FIGURE 3.2 – affichage de la comparaison des performances des algorithmes

Chapitre 4

Satisfaction de Contraintes

Les satisfaction de contraintes inclut :

- La recherche de configurations régulières
- La recherche de configurations croissantes
- La combinaison des deux types de contraintes (croissante et régulière)

4.1 Solveurs

3 solveurs ont été mis en place :

- BacktrackSolver
- MACSolver
- HeuristicMACSolver

```
Test avec BacktrackSolver:
Temps d'execution: 72 ms

BacktrackSolver a trouve une solution :
On1 = 3
On0 = -2
On3 = -1
On2 = 1
On4 = -3

Test avec MACSolver:
Temps d'execution: 444 ms

MACSolver a trouve une solution :
On1 = 4
On0 = 3
On3 = -3
On2 = -1
On4 = -2

Test avec HeuristicMACSolver:
Temps d'execution: 862 ms
```

FIGURE 4.1 – Comparaison des différents solveurs

4.2 Extraction de Connaissances

L'extraction de motifs montre :

- Les configurations fréquentes de blocs
- Les règles d'association pertinentes
- L'analyse des motifs découverts

Chapitre 5

Conclusion

Le projet démontre l'application pratique des algorithmes d'IA à un problème classique. Les différentes approches (planification, contraintes, extraction de connaissances) montrent chacune leurs avantages et limitations. Les améliorations possibles sont :

- Parallélisation des algorithmes
- Nouvelles heuristiques
- Interface graphique améliorée
- Meilleure restructuration du code